For 1.2.5 and prior Django versions, the following code should be added in your settings file *(settings.py)*:

```
CACHE_BACKEND = 'memcached://127.0.0.1:11211/'
```

For 1.3 and development versions, add the following:

```
CACHES = {
    'default' : {
        'BACKEND' : 'django.core.cache.backends.memcached.
MemcachedCache',
        'LOCATION' : '127.0.0.1:11211',
    }
}
```

Start using Memcached

From an entire website to a very tiny byte of data, almost anything can be cached. The cache in Django lets you get, add, set and delete things in the cache, referenced by a 'key'. A mechanism to delete the copy once it is expired or outdated is created by setting the *TIMEOUT* parameter.

Caching an entire website: This can be achieved by adding *django.middleware.cache.UpdateCacheMiddleware* and *django.middleware.cache.FetchFromCacheMiddleware* to *MIDDLEWARE_CLASSES* in the *settings.py* file.

Note: Remember that *'update'* middleware must be first in the list, and the *'fetch'* middleware must be last.

This is the simplest way, but the least preferred. The reason lies in the definition of caching itself. We only intend to save the results of expensive calculations so that they need not be performed the next time. Caching unnecessary things will result in an evil hack.

View cache: View is a Python function that takes a Web request and returns a Web response. Either the entire view or any part of it can be cached. Here, the value corresponding to the key is requested in the cache *{ cache.get() }* and is returned if found; else, executing the code generates the result and it gets stored in the cache.

```
from django.core.cache import cache
def my_view(request):
    cache_key = 'my_view_cache_key'
    cache_time = 1800 #time to live in seconds
    result = cache.get(cache_key)
    if not result:
        result = # some calculation for the view
        cache.set(cache_key,result,cache_time)
    return result
```

A more simple way is by using the decorator's *cache_page* that will automatically cache the view's response. It takes *cache timeout* in seconds as the argument (here is an example, in which we take 20 minutes as *cache timeout*):

```
from django.views.decorators.cache import cache_page
    @cache_page(60 * 20 )
    def my_view(request):
```

Template fragment caching: This is suitable if there is heavy processing in the template. The cache tag can be used, passing minimum parameters, as follows:

```
{% load cache %}
{% cache timeout key %}
    ....  //code that is to be cached
{% endcache %}
```

Verify the working of Memcached

Inspect the content of the cache using the Django shell: A pair of key-values is stored in the cache and we try retrieving the value from the cache corresponding to the key:

```
# python manage.py shell
>>> from django.core.cache import cache
>>> cache.set('test', 'test value')
>>> 'test' in cache
True
>>> cache.get('test')
'test value'
```

There are more ways, including *telnet*, to verify the workings of the cache.

As you have seen from the above examples, using Memcached with Django is very easy, but it is required that you cache the right data. Using it wrongly will give a negative result. To use Memcached effectively, just ponder over the following questions:

1) What is to be cached?
2) For how long is it to be cached?
3) Is it necessary to cache?

The answers will help you to improve your website's performance. END

References

[1] *http://www.djangobook.com/en/2.0/chapter15.html*
[2] *http://memcached.org/*

By: Avani M Lodaya

The author is interested in Web development, Web designing and database design. She is a FOSS enthusiast and is also interested in contributing to open source. She has been working on developing Django for over eight months. You can reach her by email at *avani9330@gmail.com*